Getting started with v2: how to start the project, step by step

Follow these steps in order. Each step says what success looks like before you move on. README.md is the overview; the details are in `docs/`.

Step	What you do	Board needed?	Time
1	Put the files on your PC	no	5 min
2	Install the tools (once)	no	1-3 h (large download)
3	Simulate: prove the Verilog works	no	15-40 min
4	Way 1: FPGA only (no software)	yes	45 min
5	Way 2: ARM + FPGA with the C program	yes	1-2 h
6	Verify and take the slide-24 measurements	yes	as needed

Step 1: Put the files on your PC

1. Download **`smart_serial_controller_v2.zip`** (the file you were sent), or from GitHub: open the repository, switch the branch to **claude/wizardly-cori-h3t0hy**, then **Code -> Download ZIP** and use the `smart_serial_controller_v2` folder inside it.
2. Windows: right-click the ZIP -> **Properties**; if there is an **Unblock** box, tick it -> **OK**.
3. Right-click -> **Extract All...** to **`C:\ssc`** (short path, no spaces).
4. Check that **`C:\ssc\smart_serial_controller_v2\README.md`** exists. If you got `C:\ssc\smart_serial_controller_v2\smart_serial_controller_v2\...`, move the inner folder up one level.

> Do not work inside Downloads, Desktop or OneDrive: Vivado has problems with spaces in folder names and with paths longer than 260 characters.

```
smart_serial_controller_v2\
  README.md, GETTING_STARTED.md   <- start here
  windows\        double-click scripts: 1_simulate, 2_build_way1, 3_build_way2, 4_open_gui,
5_noc_study
  rtl\            the Verilog (v1 engines, NoC, serial engine, crossbar, hub, DMA, top)
  boards\zedboard\  pads, top levels, pin constraints
  programs\       serial-engine programs (.se) and their assembled .hex
  sim\            testbenches (sim\unit = one per block, plus two system tests)
  sw\             C program for the ARM (Vitis)
  tools\          assembler, PC frame tool, scripts
  vivado\         Tcl scripts the .bat files run
  noc_study\      bus vs crossbar vs NoC comparison
  docs\           SPEC, register map, verification, measurements, code walkthrough,
                  one explanation per block (docs\blocks), PDFs (docs\pdf)
```

Step 2: Install the tools (once)

1. **Vivado + Vitis** (AMD Unified Installer, free AMD account; choose the product **Vitis**, which includes Vivado). Any version from 2020.2 on.
   - Devices: only **SoCs -> Zynq-7000** is needed.
   - Tick **Install Cable Drivers**. The free licence covers the XC7Z020.
   - Note the install folder: `C:\AMDDesignTools\<version>\Vivado` or `C:\Xilinx\<version>\Vivado` (2025.1+), `C:\Xilinx\Vivado\<version>` (older).
2. **ZedBoard board files** (needed for Way 2): in Vivado, **Tools -> Vivado Store... -> Boards -> Refresh -> Avnet -> ZedBoard -> Install** (Vivado 2020.x: New Project wizard -> Boards tab -> Refresh -> download).
3. **A serial terminal:** Tera Term or PuTTY.
4. **Python 3** with pyserial (`pip install pyserial`) for `tools\pc_frame.py`, which sends the slide-17 frames from the PC.
5. **Optional, recommended for fast simulation:** WSL (Ubuntu) or Linux with `sudo apt install iverilog gtkwave yosys`. The Icarus scripts run all 15 testbenches in about 15 minutes.
6. **Optional:** a logic analyser (any FX2-based 8-channel one works with PulseView/sigrok) for JD.

Linux only: run the cable-driver installer once as root (`<install>/<version>/data/xicom/cable_drivers/lin64/install_script/install_drivers/install_drivers`).

Step 3: Simulate (no board needed)

Option A, WSL/Linux with Icarus (fastest):

```
cd /mnt/c/ssc/smart_serial_controller_v2      (WSL)   or your Linux folder
sh sim/run_unit.sh          # 13 unit testbenches, ~3 min
sh sim/run_system.sh        # the 2 system testbenches, ~10 min
```

Success: every test prints `ALL n CHECKS PASSED`, then `UNIT TESTS: 13 passed` and `SYSTEM TESTS PASSED`.

Option B, Windows with Vivado only: double-click **`windows\1_simulate.bat`**. It runs the system tests and the main unit tests in XSim (15-40 min) and ends with a SUMMARY saying `passed` for each. The many Vivado `INFO` lines are normal; the testbench lines start with `[n]`, `ok` or `FAIL`.

> "Could not find Vivado": open `windows\_find_vivado.bat` in Notepad and set `set "MY_VIVADO_SETTINGS=C:\Xilinx\2025.1\Vivado\settings64.bat"` (your path, one pair of quotes around the whole NAME=value).

What each test proves and what its output must look like: **docs/VERIFICATION.md**. Waveforms: section 4 of that file.

Step 4: Way 1, FPGA only

The FPGA runs the whole v2 controller with no ARM and no software: a small boot sequencer (`rtl/top/ssc2_boot.v`) does the set-up.

4a. Build

Double-click **`windows\2_build_way1_fpga_only.bat`** (10-20 min). Success: `Way 1 build finished`, `worst setup slack (WNS)` >= 0, then `SUCCESS. Bitstream: build\standalone\zed2_top_standalone.bit`. (Linux:

`vivado -mode batch -source vivado/build_standalone.tcl`.)

4b. Prepare the board (power OFF)

1. **Boot-mode jumpers** JP7-JP11 all on the **GND** side (JTAG mode).
2. **VADJ jumper J18 on 2.5 V** (the default; the switches and buttons are constrained as LVCMOS25).
3. **Pmods** (pin 1 to pin 1):
   - **JA top row:** PmodUSBUART (its JP1 jumper on **LCL**).
   - **JB:** PmodSF3, whole 12-pin connector.
   - **JC:** PmodTMP2 in the four columns nearest the GND/VCC end (pins 3-6 and 9-12), so SCL = JC3, SDA = JC4; both address jumpers open (0x4B). If a temperature read fails later, add 4.7 kOhm pull-ups from SCL and SDA to 3.3 V.
   - **JD (optional):** logic analyser. JD1-4 = whatever SW2..0 selects; JD7-10 = copies of UART TXD, UART RXD, I2C SCL, I2C SDA.
4. **All 8 switches down.** Cables: J17 (PROG) and the PmodUSBUART to the PC, 12 V supply. Power on.

4c. Program and check

1. Vivado -> **Open Hardware Manager -> Open Target -> Auto Connect -> Program Device**, choose `build\standalone\zed2_top_standalone.bit`.
2. The blue DONE LED lights. Press **BTNC** once (reset).
3. Check, as in docs/VERIFICATION.md section 5:

   - **LD4** blinks once per second; **LD0-LD3** count records (10 per second, shown divided by 4); **LD5** toggles every 3.2 s (a batch of 32); **LD6** stays off.
   - On the PC (COMx = the PmodUSBUART's port):

     ```
     python tools\pc_frame.py COM5 01 9F 03
     sent : 01 9F 03
     reply: 20 BA 19              <- flash ID through the network (slide 17)
     ```

   - SW7 up, then `python tools\pc_frame.py COM5 05 4B 01 00 02 --temp` -> `reply: 0C 80` and your room temperature.
   - SW2..0 = 010: JD1-4 show the SPI lines; = 100: JD2 shows SM0's UART frames (the temperature bytes every 100 ms) - the slide-14 switch.

Step 5: Way 2, ARM + FPGA (the final system)

5a. Build the hardware

Double-click **`windows\3_build_way2_arm_fpga.bat`** (15-30 min). Success: `Way 2 build finished`, WNS >= 0, then `SUCCESS. Hardware for Vitis: build\ps_system\ssc2_system.xsa`. To see the block design (PS, the controller, HP0), open `build\ps_system\ssc2_ps.xpr` -> **Open Block Design**.

5b. Create the software in Vitis

Copy **all eight files** of `sw\` into the application's `src` folder: `main.c, ssc2_driver.c, ssc2_driver.h, ssc2_regs.h, se_demos.c, se_demos.h, se_programs.h, oled_font.h`.

Vitis 2023.2 or newer (Unified IDE):

1. Start Vitis with an empty workspace, e.g. `C:\ssc\vitis_ws`.
2. **File -> New Component -> Platform**: name `ssc2_platform`, hardware = `build\ps_system\ssc2_system.xsa`, standalone, `ps7_cortexa9_0`, Finish, then **Build**.
3. **File -> New Component -> Application**: name `ssc2_app`, platform `ssc2_platform`, Finish.
4. Copy the 8 files into `ssc2_app\src`, then **Build**.

Vitis 2020.2 - 2023.1 (Classic):

1. **File -> New -> Application Project**, "Create a new platform from hardware (XSA)", choose the `.xsa`, name `ssc2_app`, processor `ps7_cortexa9_0`, template **Empty Application (C)**.
2. Right-click `ssc2_app/src` -> **Import Sources** -> the `sw` folder, all 8 files. Build (hammer).

5c. Run

1. Connect **J14** (the ZedBoard's own USB-UART) too, open a terminal on its COM port at 115200 8N1.
2. Run: Unified IDE -> **Run**; Classic -> right-click `ssc2_app` -> **Run As -> Launch Hardware** (with "Program FPGA" ticked).
3. Success:

```
=== Smart Serial Controller v2 - Way 2 (ARM + FPGA) ===
controller v2 found at 0x43c00000
ssc2> info        ID 0x53534302  VERSION 0x00020000 ... JA=UART JB=SPI JC=I2C
ssc2> id          JEDEC ID: 20 ba 19 (Micron - PmodSF3)
ssc2> temp        raw 0x0c 0x80 -> ... = +25.0000 C, CPU load: ~13 accesses, 2 interrupts
ssc2> hub         batch 1: 32 records ... CPU load for 32 readings: 2 APB accesses, 1
interrupt
ssc2> switch      JD: SPI -> SM0, SW_CYCLES ..., "HELLO FROM SM0 ON JD" on JD2
ssc2> se man      Manchester: 32 bytes, 0 errors, CRC equal, PASS
ssc2> se oled HI  the OLED shows the text and the temperature
```

Every command and its correct output: docs/VERIFICATION.md section 6.

Step 6: Verify and measure

- **Checklist** (simulation, Way 1, Way 2, logic analyser): docs/VERIFICATION.md.
- **The six measurements of slide 24** (flexibility, speed, switch time, CPU load, area, NoC scalability), how to take each one and the simulation numbers to compare with: docs/MEASUREMENTS.md.
- **NoC study:** `bash noc_study/run_study.sh` (WSL/Linux) and `windows\5_noc_study_vivado.bat` (Vivado LUTs and Fmax); noc_study/README.md.

Troubleshooting

Symptom	Fix
A `.bat` says "Could not find Vivado"	set `MY_VIVADO_SETTINGS` in `windows\_find_vivado.bat` (Step 3)
"file not found" during a Windows build	path too long: use `C:\ssc\smart_serial_controller_v2`
`ZedBoard board part not found` (Way 2)	install the board files (Step 2)
`apply_bd_automation` error in Way 2	open `build\ps_system\ssc2_ps.xpr`, delete the block design `system`, run the script again from a fresh Vivado; the log line before the error names the interface
Way 1: `pc_frame.py` gets nothing	use the PmodUSBUART's COM port (not J14), check JA top row, press BTNC, SW7 down for `01 9F 03`
Way 1: LD6 on, LD0-3 do not count	the hub's TMP2 reads fail: PmodTMP2 position (JC3/JC4), address jumpers open, add pull-ups
Flash ID `FF FF FF` / `00 00 00`	PmodSF3 the wrong way round, or not on JB
Way 2 prints "reading the controller ID" and stops	the FPGA is not programmed: tick "Program FPGA" or program the bitstream first
`ERROR: ID ... / VERSION ...` (Way 2)	wrong bitstream (v1?) or address: the Address Editor must show ssc_0 at 0x43C0_0000
`hub` never prints a batch	`temp` works? (TMP2 present). `info`: hub errors / time-outs counting? The batch needs 32 good readings (3.2 s)
Vitis: `xtime_l.h` / `xiltimer.h` not found	create a *standalone* platform for `ps7_cortexa9_0`; the code picks the right header for your Vitis
`se i2c` says no ACK but `temp` works	normal if JC is held by the I2C engine mid-transfer; run it again. `se i2c` uses SM0's open-drain program on JC at 100 kHz
OLED stays dark	`se oled TEXT` powers it up (VDD, reset, charge pump, VBAT); check J18 jumper and that no other design drives the OLED pins
Timing not met (WNS < 0)	open `timing_summary.rpt`; report the failing path (the design is meant for 100 MHz)

How to learn the code (suggested order)

1. docs/CODE_WALKTHROUGH.md: every module and two end-to-end traces (a temperature reading, the flash ID).
2. docs/SPEC.md: the contract between the blocks (packet format, ports, registers).
3. The blocks, each with its explanation in `docs/blocks/`: `noc.md` (packet helpers, router, mesh) -> `xbar.md` -> `serial_engine.md` -> `se_programs.md` -> `ni_uart.md`, `ni_spi_i2c.md`, `ni_se.md` -> `hub.md` -> `dma_writer.md`.
4. `rtl/top/ssc2_core.v`: how they are wired together.
5. docs/REGISTER_MAP.md before changing the C code.

How this fits the plan on slide 25

Version (slide 25)	Demo	Where
V5 pin crossbar (1 wk)	JD switches from SPI to UART	`rtl/xbar`, `switch` command, Way 1 SW2..0
V6 serial engine (4 wk)	UART, SPI and OLED programs	`rtl/se`, `programs/`, `se ...` commands
V7 routers + interfaces (4 wk)	flash ID read through the NoC	`rtl/noc`, `flowa` + `pc_frame.py`
V8 hub + DMA writer (3 wk)	CPU sleeps, batches land in DDR	`rtl/hub`, `hub` command
V9 measurements (2 wk)	comparison tables and the report	`docs/MEASUREMENTS.md`, `noc_study/`